Deep Learning, Generative AI & Neural Networks

3-Week Intensive Learning Plan

4 hours/week · Hands-on · From scratch to deployment

Roadmap

WEEK 1

The Neuron & Backprop Engine

MILESTONE

Build a working autograd engine from scratch

Curriculum: ① ② ⑤

WEEK 2

Deep Architectures & Training Dynamics

MILESTONE

Train a CNN on real data with >90% acc and explain WHY

Curriculum: ③ ⑥ ⑦

WEEK 3

Generative AI & Full-Stack Deploy

MILESTONE

Deploy a GenAI app end-to-end

Curriculum: ④ + GenAI

7 Core Principles

The theoretical backbone of this course

1 Universal Approximation

One hidden layer approximates any function — depth is about efficiency

2 ReLU Changed Everything

Sparse activation + no vanishing gradients = trainable deep networks

3 Overparameterization Helps

More params than data, yet generalizes — implicit regularization of SGD

4 NNs Don't Understand

They minimize loss. They detect statistical structure, not meaning.

5 Backprop = Chain Rule

Efficient gradient computation via dynamic programming. Nothing mystical.

6 Geometry Matters

Flat minima → better generalization. Sharp minima → poor robustness.

7 Capacity ≠ Performance

Architecture, init, normalization, data quality dominate over depth.

WEEK 1

The Neuron & Backprop Engine

LEARNING FOCUS

How a neural network actually computes — forward pass, loss, gradients, update.
One concept: automatic differentiation is the chain rule, implemented as a graph traversal.

RESOURCES

Karpathy — "Zero to Hero" Lecture 1: micrograd
Builds autograd from scratch in 2 hours

3Blue1Brown — Neural Networks (Ch. 1-3)
Unmatched geometric intuition (~40 min)

BUILD EXERCISE

Implement micrograd yourself:

1. Scalar-valued autograd engine
2. Value class with +, *, ReLU, backward()
3. Neuron → Layer → MLP classes
4. Train on 2D moons dataset
5. Visualize decision boundary

YOU KNOW YOU'VE GOT IT WHEN...

You can draw the computation graph for L = (w·x + b).relu() and manually compute ∂L/∂w

Source: Karpathy Zero to Hero, 3Blue1Brown Neural Networks

Week 1 Key Concepts

① Universal Approximation

A single hidden layer can approximate any continuous function — but may need impractically many neurons.

Depth is about efficiency of representation, not just power.

② ReLU Changed Everything

Before: sigmoid/tanh → vanishing gradients.
ReLU: sparse activation + gradient is 0 or 1.

This made deep networks trainable.

⑤ Backpropagation = Chain Rule

Nothing mystical. Given $L = f(g(h(x)))$, the chain rule gives:

$$\partial L/\partial x = (\partial L/\partial f) \cdot (\partial f/\partial g) \cdot (\partial g/\partial h) \cdot (\partial h/\partial x)$$

Backprop traverses the computation graph in reverse, accumulating gradients. It is dynamic programming applied to differentiation.

WEEK 2

Deep Architectures & Training Dynamics

LEARNING FOCUS

Going from toy to real. Architecture + initialization + normalization matter more than raw depth.

RESOURCES

PyTorch tutorials — Training a Classifier
Canonical intro, short and focused

Karpathy — makemore Part 2
BatchNorm, init, training dynamics

d2l.ai Chapter 7 — CNNs
Runnable code + geometric explanations

BUILD EXERCISE

CNN Ablation Study on CIFAR-10:

1. Start with bad arch (sigmoid, no BN)
2. Switch to ReLU → measure gain
3. Add He initialization → measure
4. Add BatchNorm → measure
5. Add data augmentation → measure
6. Log all experiments, plot curves

Target: >90% test accuracy

YOU KNOW YOU'VE GOT IT WHEN...

You can look at a training curve and diagnose: learning rate too high? Vanishing gradients? Overfitting?

Source: PyTorch Tutorials, Dive into Deep Learning

Week 2 Key Concepts

③

Overparameterization Helps

Networks with more parameters than data points generalize well. SGD acts as implicit regularizer — it finds simple solutions in weight space.

⑥

Geometry Matters

Training navigates a high-dimensional loss landscape. Flat minima generalize better. Sharp minima break under distribution shift.

⑦

Capacity ≠ Performance

More layers ≠ better. He/Xavier init, BatchNorm, data quality, and architecture design dominate raw capacity.

Why Initialization Matters

Same architecture, same data, same optimizer — only the initial weights change:

Method	Formula	Best For	Result
Random N(0,1)	`W ~ N(0, 1)`	Nothing	Exploding activations
Xavier / Glorot	`W ~ N(0, 2/(n_in+n_out))`	Sigmoid, Tanh	Stable gradients
He / Kaiming	`W ~ N(0, 2/n_in)`	ReLU	Optimal gradient flow

The lesson: same capacity, vastly different outcomes. Architecture choices are first-order.

WEEK 3

Generative AI & Full-Stack Deployment

LEARNING FOCUS

From discriminative to generative. Generative models learn the data distribution. Quality depends on what they saw, not what they know.

RESOURCES

HuggingFace NLP Course — Ch. 1-3
Fastest path to using pretrained transformers

fast.ai — Lesson 9: Stable Diffusion
Diffusion models without drowning in math

Simon Willison's blog
Practical, opinionated LLM building guides

BUILD EXERCISE

Full-Stack GenAI Application:

1. Fine-tune / prompt-engineer a model
2. Domain: medical procedure summaries
3. Build Gradio interface with controls
4. Deploy to HuggingFace Spaces
5. Explain every layer of the stack

YOU KNOW YOU'VE GOT IT WHEN...

You can explain the difference between a model that 'understands' and one that learned statistical patterns

Source: HuggingFace Course, fast.ai

④ Neural Networks Don't "Understand"

What they DO

- Minimize a loss function
- Detect statistical regularities
- Approximate P(output | input)
- Interpolate between training examples
- Compress information into weights

What they DON'T

- Reason about truth or falsehood
- Grasp causation (only correlation)
- Verify their own outputs
- Know what they don't know
- Have intentions or beliefs

This distinction is critical for safety-sensitive domains like medical AI.
A model that has "seen" medical text is not a model that "knows" medicine.

Projects Ladder

P1 **micrograd: Autograd from Scratch**

Week 1 | ★★☆☆☆

"I built backpropagation. Here's gradient flowing backward."

P2 **CIFAR-10 CNN Training Lab**

Week 2 | ★★★☆☆

"I trained an image classifier to 92% accuracy. Here's why BatchNorm + He + ReLU won."

P3 **Generative Text Application**

Week 3 | ★★★★☆

"I deployed an app that generates medical summaries. Try it."

P4 **NN Myths Blog Post (Bonus)**

Bonus | ★★★☆☆

Debunk 5 common NN misconceptions with code demos.

5 Common Traps

1

Tutorial Hell

Never watch >30 min without coding. Pause, implement, continue.

2

Skipping Math That Matters

Hand-compute one backward pass. You don't need measure theory, but you DO need gradients.

3

Blaming the Model

Inspect data first: visualize samples, check balance, find label noise. 10 min of EDA saves 10 hours.

4

Overcomplicating Architecture

Start with the simplest model. Add complexity only when you can measure the improvement.

5

Confusing 'It Works' with 'I Understand'

After every experiment, write one paragraph explaining WHY. If you can't, go back.

Proving You Learned This

PORTFOLIO PIECES

- GitHub repo with micrograd + training notebooks
- Experiment tracking via Weights & Biases (free tier)
- Deployed Gradio app on HuggingFace Spaces

CERTIFICATIONS

- DeepLearning.AI Specialization (Coursera)
- HuggingFace Certification

COMMUNITIES

fast.ai forums
Most helpful DL learning community

r/MachineLearning & r/learnmachinelearning
Discussion, papers, project feedback

HuggingFace Discord
Active, welcoming, GenAI-focused

Today's Assignment

30 minutes to build momentum

Build a Single Neuron in Pure Python

1. Implement a Value class (data + grad)
2. Support __add__, __mul__, __neg__
3. Implement sigmoid: 1 / (1 + exp(-x))
4. Compute: y = sigmoid(w1*x1 + w2*x2 + b)
   w1=0.5, w2=-0.3, b=0.1, x1=2.0, x2=3.0
5. Call y.backward() and inspect gradients

This is the seed of your Week 1 project.

Accompanying Notebooks

SERIES 1: PURE PYTHON

Only Python + NumPy + matplotlib. No frameworks.

NB 01 The Neuron & Backprop Engine

Value class, autograd, MLP, moons training

NB 02 Training Dynamics

Matrix MLP, MNIST, init experiments, loss landscape

NB 03 Generative Models from Scratch

Char-level LM, autoencoder, temperature sampling

SERIES 2: PYTORCH

Professional framework. Build on pure-Python intuition.

NB 04 PyTorch Fundamentals & Autograd

Tensors, autograd, nn.Module, moons training

NB 05 CNNs & Overparameterization

CIFAR-10, ablation study, feature visualization

NB 06 Generative AI with Transformers

Attention, mini-GPT, HuggingFace, Gradio deploy

Start Building.
Not Watching.

"Every concept introduced by building something, not reading about it."

3 weeks · 6 notebooks · 4 projects · 1 deployed app